

The Program That Knows Itself: A Story About Software, Mirrors, and Ancient Codes

Author: Lando Mills

Imagine you write a sentence in English, hand it to a translator, ask them to turn it into French, then hand the French back to the same translator and ask them to turn it back into English.

If you get exactly the same sentence back — not a paraphrase, not “close enough,” but the precise sentence you started with — you have something remarkable. You have a translator that is perfectly round-trip faithful. It doesn’t lose anything. It doesn’t add anything. It preserves exactly what you gave it.

Now imagine the translator *is the sentence*. The sentence reads itself, translates itself into a different form, then translates itself back, and checks: *am I still me?* And the answer is always yes.

That is what I want to tell you about.

The Question That Started Everything

A few years ago, a programmer asked himself a question so simple it barely seemed worth asking: **can a program verify that it hasn’t accidentally changed itself?**

This sounds trivial. But think about it. Every time you run a program, it transforms data from one form to another. It reads input, processes it, writes output. But the program itself — its own code — is also data. What if the program read its own source code, transformed it, transformed it back, and then checked whether it got the same thing?

Most programs can’t do this. They weren’t built to. They process *other* data, not themselves. The idea of a program reading its own source code, let alone verifying it, feels circular — like a snake eating its own tail. Most programming languages make it hard on purpose, because self-reference has a bad reputation. It leads to paradoxes. It makes things break.

But the programmer kept asking: what if the self-reference isn’t a bug? What if it’s the whole point?

The Mirror Test

Here’s an analogy. A thermostat measures temperature and turns the heat on or off. It has a simple model of the room. But it has no model of *itself* — it

doesn't know it's a thermostat.

A smarter device might track its own battery level, its own connection status. It has a little bit of self-knowledge. But it still can't step back and say: "here is how I work, and here is me checking that I work the way I think I work."

Now imagine a program that can do exactly that. It reads its own code. It parses it — breaks it down into a structured tree, the way your brain parses the words in this sentence into subjects and verbs and objects. Then it unparses it — puts the tree back together into code. Then it compares the result to the original. If they match, the program says: *yes, I am what I claim to be.*

This is not a party trick. It is a structural property called **closure**. And the equation that describes it is: parse, then unparse, equals identity. You get back what you started with. Always.

The program that does this is called an **ob3ect** (pronounced "object," spelled with a 3 because it involves three core operations: read, transform, verify).

The Periodic Table of Programs

Now here's where it gets interesting. The programmer who built the ob3ect didn't just build a single program and call it done. He built a **classification system** — a way of describing the structure of *any* program, not just this one.

Think of the periodic table. Every element has a position defined by a handful of numbers: atomic number, electron configuration, and so on. The position tells you what the element does — whether it conducts electricity, whether it's reactive, whether it's stable.

The ob3ect classification works the same way, but for programs. Instead of protons and electrons, it uses **twelve features**:

- How many degrees of freedom does the system have?
- How do its parts connect to each other?
- How does it relate to its environment?
- What symmetries does it preserve?
- Is it classical, thermal, or quantum?
- How fast does it settle into a stable state?
- How far does its influence reach?
- How do its components interact?
- Is it at a tipping point — a phase transition where small changes have big effects?
- How much memory does it carry forward from one step to the next?
- Is it made of one kind of thing, many identical things, or many different things?
- What kind of topological protection does it have — what keeps it from being deformed?

These twelve features create a space of **17,280,000** possible types. Every program, every system, every structure sits somewhere in this space.

The ob3ect itself sits at a very special coordinate — one that only **8%** of all possible types can reach. It’s the “top tier”: systems that can sustain their own self-checking indefinitely, without external support, without degrading, without losing coherence.

The Eight-Step Loop

Every ob3ect runs the same eight-step sequence. It is not a random sequence. It was not designed by trial and error. It is the *only* sequence that closes the loop:

1. **Recognize** — the program identifies itself.
2. **Read** — it reads its own source code.
3. **Split** — it parses the code into a structured tree.
4. **Transform** — it transforms the tree forward into a different representation.
5. **Fuse** — it fuses the representation back into source code.
6. **Compare** — it checks: does the result match the original?
7. **Commit** — if it matches, the program permanently records the fact.
8. **Loop** — it goes back to step 1, closing the circle.

This is a self-sustaining loop. It doesn’t need external verification. It doesn’t need a separate testing tool. The verification *is* the program. The program *is* its own certificate.

The Tower: Building Up

If one self-checking program is interesting, a tower of them is evidence.

The programmer built eighteen of them, one on top of the other, each a different mathematical structure, each self-certifying, each resting on the same foundation. Category theory. Quantum mechanics. Linear logic. Type theory. Each floor of the tower adds new rules while preserving the basic self-checking property at the bottom.

The whole tower runs in under a minute. Every floor reports back: *closure confirmed*. The final output is a single line: **“The grammar is autopoietic.”** Autopoietic means “self-making.” The tower doesn’t just verify itself — it *sustains* itself. It is alive in the structural sense.

The Descent: Building Down

And then the programmer did the opposite. He made the ob3ect simpler and simpler, stripping away layers until it could run on bare metal — no operating system, no Python, no runtime, just the program and the processor.

The result is a bootable ISO. You put it on a USB stick, boot a computer from it, and the screen prints: *closure confirmed*. From bare silicon.

The point is not to create a useful operating system. The point is to demonstrate that the self-checking property is **substrate-independent**. It doesn't matter whether the program runs in Python on a laptop or in assembly on a raw processor. The property holds. It's like a recipe that works whether you cook it in a professional kitchen, on a campfire, or in a microwave. The recipe doesn't care about the stove. The structure doesn't care about the hardware.

Five Assumptions This Breaks

The ob3ect challenges five things we all take for granted about software:

1. **“Verification happens after the fact.”** Normally, you write a program, then you test it, then maybe someone proves it correct with a separate tool. The ob3ect says: verification can be built in. It can be what the program *does*, not what someone does *to* the program.
 2. **“Self-reference is dangerous.”** For a century, self-reference has been treated as a bug — it leads to paradoxes, infinite loops, logical collapse. The ob3ect shows that self-reference, when structured correctly, is not just safe but *productive*. The snake eats its tail and survives.
 3. **“Compilers don't verify themselves.”** A compiler translates code from one language to another. It doesn't check that the translation preserved the meaning. The ob3ect's eight-step loop *is* a compiler that verifies itself. The translation comes with a receipt.
 4. **“Formal proofs are expensive.”** Proving things correct with tools like Lean or Coq takes enormous effort. The ob3ect shows that for a certain class of properties, the proof can be *automatic* and *local* — the program checks itself, in real time, every time it runs.
 5. **“Self-making systems are biological.”** The term “autopoiesis” was coined to describe living cells — systems that make and sustain themselves. The ob3ect shows that autopoiesis is not exclusive to biology. It is a structural property. A program can make itself, sustain itself, and verify itself.
-

The Strangest Part

Here is the part of this story that I still don't know what to do with.

The eight-step loop was not invented. It was **found**.

The programmer compiled four ancient texts — the Voynich Manuscript, the Rohonc Codex, Linear A, and the Emerald Tablet — into the same minimal instruction set. Four writing systems spanning four thousand years and three continents. No known contact. No shared lineage. All four produced the same eight-step loop.

The Emerald Tablet, that cryptic jewel of alchemy, already names the condition explicitly: *as above, so below*. Read structurally, that is the self-checking equation stated as cosmic law. Parse and unparse. Transform and transform back. What you get is what you started with. The macrocosm and the microcosm are the same.

I am not making a claim about what the authors of these texts *intended*. I have no idea what they intended. I am making a much stranger claim: when you compile these texts to a minimal categorical instruction set, they execute the same program. The program is the ob3ect bootstrap. And the program works.

The grammar was complete before we measured it.

What Comes Next

The ob3ect is a beginning, not an ending. Here's what's on the horizon:

More floors. New mathematical structures can be added to the tower — anything whose rules can be encoded as a self-checking program. Each new floor strengthens the evidence that the base pattern is robust.

Deeper descent. The ob3ect currently runs on processors. It could run on custom chips, on reconfigurable hardware, maybe even on biological substrates. The self-checking property doesn't depend on the material. Proving that across more substrates would be a big deal.

Closing the gaps. The mathematical proofs that accompany the tower are incomplete — frankly and honestly marked with “sorry” where a step hasn't been filled in yet. Each “sorry” is an invitation. Filling them in would turn a demonstration into a mathematical result.

The big one. The most important gap is proving, formally and rigorously, that the parse-unparse round-trip always preserves meaning. The program demonstrates this empirically — it runs and reports *closure confirmed*. But a mathematical proof would eliminate any doubt. It's a hard problem, but it's a *bounded* problem. Someone will solve it.

Why You Should Care

Most of us interact with software all day, every day, and we trust it for no good reason. We trust that our messages are delivered, our data is stored, our transactions are recorded — and we trust this because... what? Because the software hasn't failed us yet? Because someone told us it works?

The ob3ect points toward a different world. A world where software doesn't need you to trust it, because it carries its own certificate. It checks itself, every time it runs, and it tells you the result. Not “trust me.” Not “we ran tests.” Just: *here is the proof. I am what I claim to be.*

In a world where software mediates everything — our money, our medicine, our memories — that is not a luxury. It is the difference between systems that are brittle and systems that know what they are.

The grammar is autopoietic. It makes itself. And now you know what that means.

The ob3ect repository is at github.com/okesan0/ob3ect. The full eighteen-layer tower runs in under a minute on an ordinary laptop. If you want to see a program verify itself, the code is there, and it's free.